

Anno Accademico 2025/2026

Leonardo Donati

Appunti
Intelligenza Artificiale

Teoria.



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Indice

1	Introduzione	4
1.1	Ripasso Algebra Lineare	4
1.2	Ripasso Statistica	4
1.3	Machine Learning	5
2	Apprendimento Supervisionato	6
2.1	Dati	6
2.2	Processo di Apprendimento Supervisionato	6
2.3	Apprendimento	7
2.3.1	Misure di valutazione	7
2.3.2	Modello di valutazione	7
2.4	Iperparametri	7
2.5	Altre Misure di Classificazione	8
3	Probabilistic Classifier and Linear Classification Models	9
3.1	Definizioni	9
3.1.1	Formula di Bayes	9
3.1.2	Maximum Likelihood	9
3.1.3	Maximum A-Posteriori	10
3.2	Classificatore Ottimo di Bayes	10
3.2.1	Stima della densità	10
3.2.2	Naive Density (Naive DE)	10
3.3	Linear Discriminant Analysis (LDA)	11
3.3.1	Quadratic Discriminant Analysis	11
3.4	Logistic Regression	12
3.4.1	Ottimizzazione dei Parametri	12
3.4.2	Discesa del Gradiente	13
4	Support Vector Machine	14
4.1	Classificazione Binaria	14
4.1.1	Problema quadratico SVM	15
4.1.2	Ottimizzazione Vincolata	15
4.1.3	Classificatore SVM	16
4.1.4	SVM non Lineare	16
5	Ensembles	18
5.1	Bagging	18
5.2	Boosting	19
5.3	AdaBoost	19
6	Random Forest	21
6.1	Alberi Binari	21
6.2	Ottimizzazione	21
6.3	Foresta	22

7	Clustering	23
7.1	Distanza e Similiarità	23
7.2	Clustering Gerarchico	23
7.2.1	Strategia di Merging	24
7.3	Clustering non Gerarchico	24
7.4	Grafi	24
7.4.1	Operatori	25
7.5	Partizione Dati tramite Grafi	26
7.5.1	Bi-Partizione del Grafo	26
7.5.2	Minimum Cut	26
8	Dimensionality Reduction	28
8.1	Multi Dimensional Scaling (DMS)	28
8.1.1	PCA	28
8.1.2	Riduzione non Lineare	29
8.1.3	LLE e Laplacian Eignmap	29

Capitolo 1

Introduzione

1.1 Ripasso Algebra Lineare

DEFINITION Spazio vettoriale

Lo **spazio vettoriale** $\mathbb{R}^n$ è un insieme di elementi $x = [x_1, \dots, x_n]$ where each $x_i \in \mathbb{R}$. Gli elementi x sono chiamati **vettori** e soddisfano queste proprietà:

1. **Addizione:** Se $x \in \mathbb{R}^n$ e $y \in \mathbb{R}^n$, allora $x + y = [x_1 + y_1, \dots, x_n + y_n] \in \mathbb{R}^n$.
2. **Prodotto scalare:** Se $x \in \mathbb{R}^n$ e $a \in \mathbb{R}$ allora, $a \cdot x = [a \cdot x_1, \dots, a \cdot x_n]$.
3. **Prodotto interno:** Se $x \in \mathbb{R}^n$ e $y \in \mathbb{R}^n$ allora, $x \cdot y = \sum_{i=1}^n x_i \cdot y_i$.

DEFINITION Matrice

Una **matrice** $X \in \mathbb{R}^{n \times m}$ è un vettore rettangolare di elementi $x_{ij} \in \mathbb{R}$, $1 \leq i \leq n$, $1 \leq j \leq m$:

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{pmatrix}$$

Deve supportare queste operazioni:

1. **Addizione:** Se $X \in \mathbb{R}^{n \times m}$, $Y \in \mathbb{R}^{n \times m}$ e $Z = X + Y$, allora $Z \in \mathbb{R}^{n \times m}$ e $Z_{ij} = X_{ij} + Y_{ij}$.
2. **Prodotto scalare:** Se $X \in \mathbb{R}^{n \times m}$, $a \in \mathbb{R}$ e $Z = a \cdot X$, allora $Z \in \mathbb{R}^{n \times m}$ e $Z_{ij} = a \cdot X_{ij}$.
3. **Prodotto fra matrici:** Se $X \in \mathbb{R}^{n \times m}$, $Y \in \mathbb{R}^{m \times n}$ e $Z = X \cdot Y$, allora $Z \in \mathbb{R}^{n \times n}$ e $Z_{ij} = \sum_{k=1}^m X_{ik} \cdot Y_{kj}$.

1.2 Ripasso Statistica

DEFINITION Distribuzione di Probabilità

Una distribuzione di probabilità P su uno spazio campionario Ω è un'applicazione dai sottoinsiemi di Ω ai numeri reali che soddisfa le seguenti condizioni:

- **Non-negatività:** $P(\alpha) \geq 0$, $\forall \alpha \subseteq \Omega$.
- **Normalizzazione:** $P(\Omega) = 1$.
- **Additività:** $\forall \alpha, \beta \subseteq \mathbb{R}$ che sono insiemi disgiunti, $P(\alpha \cup \beta) = P(\alpha) + P(\beta)$.

1.3 Machine Learning

DEFINITION Machine Learning

Il **machine learning** è un campo di studio che conferisce ai computer la capacità di apprendere senza essere esplicitamente programmati.

Esistono due tipi di approcci:

1. **Non parametrico:** l'apprendimento avviene memorizzando i dati di addestramento (memorizzazione).
2. **Parametrico:** l'apprendimento avviene utilizzando un algoritmo per adattare i parametri di un modello matematico o statistico sulla base di dati di addestramento. Per esempio:

$$f_{\theta}(x) = \sum_{d=1}^D \theta_d x_d$$

Capitolo 2

Apprendimento Supervisionato

2.1 Dati

Un insieme di record di dati $X \in X^k$, chiamati anche **esempi**, **istanze** o **casi** sono descritti da:

- k attributi (**Features**): $x = x_1, x_2, \dots, x_k$.
- Una **classe**: ogni esempio è etichettato con una classe target predefinita $c \in C$.

L'obiettivo è imparare un modello di classificazione dai dati che possono essere usati per predire la classe di nuovi casi.

2.2 Processo di Apprendimento Supervisionato

Il processo è diviso in due step:

1. Imparare un modello usando idati dell'allenamento.
2. Testare il modello con dati mai usati per vederne l'**accuratezza**.

$$\text{Accuratezza} = \frac{\text{Numero di classificazioni corrette}}{\text{Numero totali di casi}}$$

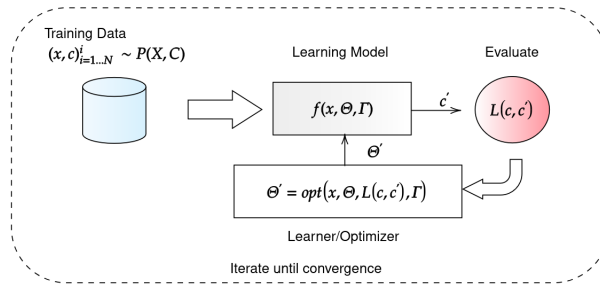


Figura 2.1: Training pipeline.

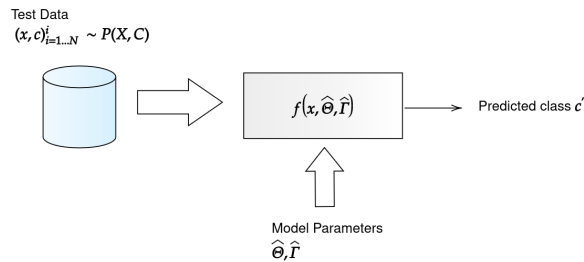


Figura 2.2: Inference pipeline.

2.3 Apprendimento

DEFINITION Apprendimento

Dati un dataset D , una classe compito C e una funzione di prestazione L , si dice che un sistema informatico apprende il compito C dai dati D se, dopo aver osservato D , la prestazione misurata tramite L migliora fino a raggiungere un livello superiore a quello di una scelta casuale.

In altre parole, i dati aiutano il sistema a migliorare le proprie prestazioni rispetto all'assenza di apprendimento.

ASSUNZIONE FONDAMENTALE I.I.D

L'apprendimento è possibile solo se il training e l'inferenza sono fatti usando dati dalla stessa distribuzione $x \sim P(X)$. Detto questo, i campioni usati sono indipendenti e identicamente distribuiti (i.i.d), dal training all'inferenza.

2.3.1 Misure di valutazione

Oltre all'accuratezza, che abbiamo visto in precedenza, ci sono molti altri metodi per valutare:

- **Efficienza:**
 - Tempo usato per costruire il modello.
 - Tempo per usare il modello.
- **Robustezza:** gestire il rumore e valori mancanti.
- **Scalibilità:** efficienza in database basati su dischi.
- **Interpretabilità:** comprensibilità e approfondimenti forniti dal modello.
- **Compattezza del modello:** dimensione dell'albero o numero di regole.

2.3.2 Modello di valutazione

Un modello usato è l'**Holdout Set**, dove il dataset D a disposizione viene diviso in due insiemi disgiunti:

- **Insieme per il training**, T_r .
- **Insieme per l'inferenza**, T_e .

NOTA BENE

L'insieme di addestramento non deve essere utilizzato per i test e l'insieme di test non deve essere utilizzato per l'apprendimento. Un insieme di test non visto fornisce una stima imparziale dell'accuratezza.

Questo metodo è solitamente usato quando i dataset D sono molto larghi.

2.4 Iperparametri

Tutti i classificatori presentano parametri aggiuntivi, denominati **iperparametri** e non vengono appresi, bensì selezionati in fase di progettazione prima dell'addestramento.

Diverse configurazioni degli iperparametri possono modificare in modo significativo le prestazioni del modello. Viene usata un'ulteriore suddivisione del dataset D , utilizzata per la messa a punto degli iperparametri e per la selezione del modello migliore, chiamato **insieme di validazione**.

EXAMPLE Train-Validation-Test

- Dato un dataset D , partizioniamo randomicamente i dati in un insieme di allenamento T_r , uno per la validazione V e uno per il test T_e . Di solito con 60/20/20, 80/10/10 o simile.
- I modelli M_i imparano su T_r per ogni scelta dell'iperparametro H_i .
- L'errore di validazione Val_i di ogni modello M_i viene valutato sull'insieme V .
- Vengono selezionati gli iperparametri H_* che presentano il valore più basso di Val_i e il classificatore viene riaddestrato utilizzando tali iperparametri sull'insieme $T_r + V$, ottenendo un modello finale M_* .
- La capacità di generalizzazione viene stimata valutando l'errore o l'accuratezza di M_* sui dati dell'insieme di test T_e .

EXAMPLE k-fold Crossvalidation

Il metodo viene spesso utilizzato quando il dataset non è sufficientemente ampio da consentire di ottenere un test set consistente.

- Partizioniamo randomicamente un dataset D in un insieme di K blocchi $B_1, \dots, B_K$.
- Per ogni fold di cross-validation $k = 1, \dots, K$:
 - Assegna $T_e = B_k$ e $T_r = D - B_k$ (i rimanenti $K - 1$ blocchi).
 - Impara il modello M_k su T_r e testalo su B_k .
- Il processo produce k accuratze.
- La stima finale delle prestazioni del modello è la media delle K accuratze.

2.5 Altre Misure di Classificazione

L'accuratezza non è l'unica misura e in alcuni casi non è nemmeno utilizzabile. Per questo vengono usate anche misure:

- **Precisione:** è il numero di esempi positivi classificati correttamente diviso per il numero totale di esempi classificati come positivi.

$$p = \frac{TP}{TP + FP}$$

- **Recall:** è il numero di esempi positivi correttamente classificati diviso per il numero totale di esempi positivi effettivi nell'insieme di test.

$$r = \frac{TP}{TP + FN}$$

- **F1-score:** è la media armonica di precisione e recall. Dà maggior peso al valore più basso.

$$\frac{1}{F1} = \frac{1}{p} + \frac{1}{r}$$

DEFINITION Cross-Entropy

La cross-entropy è una misura della teoria dell'informazione relativa alla codifica dei messaggi utilizzando il numero di bit. Valuta il numero medio di bit per scoprire un dato codificato da una distribuzione Q mentre quello originale era P .

$$\mathbb{E}_P[l_i] = \sum_x P(x) \log_2 \frac{1}{Q(x)} = - \sum_x P(x) \log_2 Q(x)$$

Capitolo 3

Probabilistic Classifier and Linear Classification Models

3.1 Definizioni

DEFINITION Classificazione

Dato un vettore feature $x \in \mathbb{R}^D$ che descrive un oggetto appartenente a una delle classi C dell'insieme $\mathcal{Y}$, predire a quale classe appartenga l'oggetto.

DEFINITION Apprendimento di classificatori

Dato un dataset di coppie di esempi $D = \{(x_i, y_i), i = 1 : N\}$ dove $x \in \mathbb{R}^D$ è un vettore feature e $y_i \in \mathcal{Y}$ è un'etichetta della classe, imparare una funzione $f : \mathbb{R}^D \rightarrow \mathcal{Y}$ che predice accuratamente l'etichetta della classe y per ogni vettore feature x .

Dato queste definizioni ed elementi, si possono definire anche due valori:

1. **Tasso di errore di classificazione:** $Err(f, D) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i \neq f(x_i))$
2. **Tasso di accuratezza della classificazione:** $Acc(f, D) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = f(x_i))$

3.1.1 Formula di Bayes

$$P(Y = c | X = x) = \frac{P(X = x | Y = c)P(Y = c)}{\sum_{c' \in \mathcal{Y}} P(X = x, Y = c')} = \frac{\phi_c(x)\pi_c}{\sum_{c' \in \mathcal{Y}} \phi_{c'}(x)\pi_{c'}}$$

Dove:

- **Funzione di probabilità a priori:** $P(Y = c) = \pi_c$ definisce la probabilità, che pescando a caso da un dataset D , ottengo una classe c .
- **Funzione di likelihood:** $P(X = x | Y = c) = \phi_c(x)$ definisce la probabilità, che nel dataset delle classi c , pescando un x_i a caso, ottengo x .

3.1.2 Maximum Likelihood

Dato un dataset etichettato $D = (x_i, y_i)_{i=1}^N$ con k classi $c_i | i = 1 \dots k$ e un campione di test generico $\hat{x}$:

EXAMPLE Approccio

- $P(x | c_i)$, trovare una funzione di probabilità p , per ogni classe c_i .
- $\hat{c} = \operatorname{argmax}_c P(\hat{x} | c)$ dato un $\hat{x}$ esterno dal dataset D , trovare la classe c_i , per cui la probabilità che $\hat{x}$ appartenga a c_i è maggiore.

3.1.3 Maximum A-Posteriori

Dato un dataset etichettato $D = (x_i, y_i)_{i=1}^N$ con k classi $c_i \mid i = 1 \dots k$ e un campione di test generico $\hat{x}$:

EXAMPLE Approccio

- $P(x \mid c_i)$, trovare una funzione di probabilità p , per ogni classe c_i .
- $\hat{c} = \operatorname{argmax}_c P(c \mid \hat{x})$ dato un $\hat{x}$ esterno dal dataset D , trovare la classe c , per cui si ha la probabilità maggiore.

3.2 Classificatore Ottimo di Bayes

Usa la funzione di classificazione:

$$f_B(x) = \arg \max_{c \in \mathcal{Y}} P(Y = c \mid X = x) = \arg \max_{c \in \mathcal{Y}} \phi_c(x) \pi_c$$

E come **errore atteso**:

$$1 - \mathbb{E}_{P(X=x)} \left[\max_{c \in \mathcal{Y}} P(Y = c \mid X = x) \right]$$

Ci sono, però, alcuni problemi, come di casi (x, y) che non sono mai testati (in casi con db piccoli) oppure il fatto che possano esistere classi e feature. La soluzione è calcolare la probabilità $P(x \mid c)$

3.2.1 Stima della densità

Si sono due ricette per stimare la densità:

1. **Maximum Likelihood parametrico**, ossia trovare una funzione di densità, con parametri stimati a:

$$\hat{\Theta} = \arg \max_{\Theta} \prod_{x_i \in D} p(x_i \mid \Theta) = \arg \max_{\Theta} \sum_{x_i \in D} \log(p(x_i \mid \Theta))$$

2. **Stima non parametrica**, partendo da un istogramma con il numero di volte in cui compare x , normalizzare l'istogramma per ottenere le probabilità di x .

3.2.2 Naive Density (Naive DE)

DEFINITION Stima Naive Density

Uno stimatore di densità che assume che tutte le variabili casuali (attributi) di $x \in \mathbb{R}^k$ siano indipendenti.

$$P(x_1, x_2, \dots, x_k \mid c) = \prod_{i=1}^k P(x_i \mid c)$$

Il classificatore **Naive Bayes** approssima il classificatore Bayesiano ottimale utilizzando una forma semplice per la funzione $\phi_c(x)$:

$$\phi_c(x) = p(X = x \mid Y = c) = \prod_{d=1}^k p(X_d = x_d \mid Y = c) = \prod_{d=1}^k \phi_{cd}(x_d)$$

La forma finale è:

$$f_{NB}(x) = \arg \max_{c \in \mathcal{Y}} \pi_c \prod_{d=1}^k \phi_{cd}(x_d)$$

Invece, per stimare c si usa un approccio a priori, senza guardare x , considero solo la probabilità che ho di trovare una classe c :

$$\pi_c = \frac{1}{n} \sum_{i=1}^n [y_i = c]$$

DEFINITION Boundary Conditions

Sono delle rette che dividono il nostro insieme di partenza, in varie porzioni, ognuno per la sua retta. Equivalgono al punto in cui l'equazione è $= 0$.

Nel caso di due sole classi, esce un'equazione quadratica:

$$\sum_{d=1}^D (a_d x_d^2 + b_d x_d) + c = 0$$

Nel caso di classi multiple, l'equazione è quadratica a tratti.

NOTE FINALI

- Il classificatore Naive, è relativamente veloce e facile da implementare.
- Richiede pochi parametri $O(DC)$.
- Si basa sulla condizione di indipendenza (Naive condition), raramente rispettata nel mondo reale e porta a una precisione minore.

3.3 Linear Discriminant Analysis (LDA)

Con LDA, si assume una comune covarianza tra le feature, non si assume quindi l'indipendenza.

La funzione di classificazione è:

$$f_{LDA}(x) = \arg \max_{c \in \mathcal{Y}} \phi_c(x) \pi_c$$

Dove:

$$\phi_c(x) = \mathcal{N}(x; \mu_c, \Sigma) = \frac{1}{|2\pi\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_c)^T \Sigma^{-1} (x - \mu_c) \right)$$

Come per Naive Bayes, i parametri della LDA vengono appresi utilizzando un approccio maximum likelihood, il che si riduce all'impiego di stime campionarie:

- **Probabilità di classe:** $\pi_c = \frac{1}{n} \sum_{i=1}^n [y_i = c]$
- **Significato di classe:** $\frac{\sum_{i=1}^n [y_i=c] x_i}{\sum_{i=1}^n [y_i=c]}$
- **Covarianza comune:** $\Sigma = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_{y_i})(x_i - \mu_{y_i})^T$

Se le classi hanno la stessa matrice di covarianza, allora le boundary conditions sono lineari in x . Quindi, LDA è un classificatore lineare. Con la covarianza, abbiamo ottenuto 2 bordi distinti, ognuno per ogni classe. Questo ci permette di evitare problemi di sovrapposizione e mascheramento.

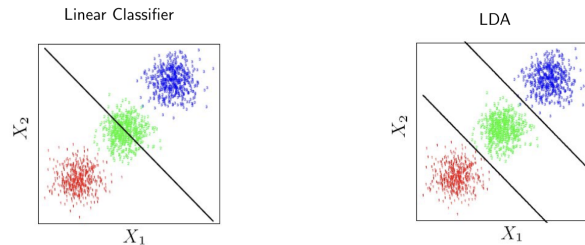


Figura 3.1: Boundary Conditions LDA.

3.3.1 Quadratic Discriminant Analysis

Se durante la scelta della covarianza, si sceglie un modello diverso, è possibile ottenere un modello QDA, un modello non lineare.

Un modello LDA, risulta spesso più robusto, ma l'assunzione di linearità è spesso non rispettata nei casi reali. Un modello QDA, invece è più dinamico, ma è più difficile controllare overfitting e altri problemi.

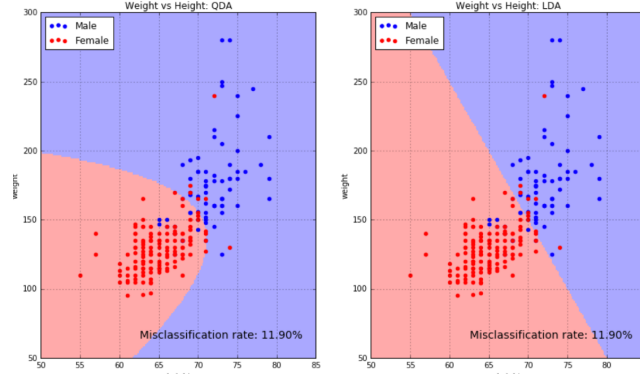


Figura 3.2: Confronto fra LDA e QDA.

NOTE FINALI

- La dipendenza quadratica da D rende l'LDA più lento del Naive Bayes di un fattore D , sia in fase di apprendimento che di classificazione.
- Il modello richiede $O(D^2C)$ parametri. Ciò può comunque rappresentare una buona compressione dei dati quando $D \ll N$.
- In generale, la LDA richiederà più dati rispetto a Naive Bayes, poiché deve stimare i parametri $O(D^2)$ nella matrice di covarianza aggregata Σ .

3.4 Logistic Regression

La logistic regression è un classificatore discriminativo probabilistico A-Posteriori. La sua funzione di classificazione è:

$$f_{LR}(x) = \arg \max_{c \in \mathcal{Y}} P(Y = c | x)$$

Nel caso binario, modella direttamente la probabilità a posteriori come:

$$P(Y = 0 | x) = \frac{e^{w^T x + b}}{1 + e^{w^T x + b}}$$

$$P(Y = 1 | x) = \frac{1}{1 + e^{w^T x + b}}$$

Calcolando le boundary conditions, si dimostra che il che è lineare. C'è anche il caso con K classi:

$$P(Y = c | x) = \frac{\exp(w_c^T x + b_c)}{1 + \sum_{l=1}^{K-1} \exp(w_l^T x + b_l)}$$

$$P(Y = K | x) = \frac{1}{1 + \sum_{l=1}^{K-1} \exp(w_l^T x + b_l)}$$

3.4.1 Ottimizzazione dei Parametri

I parametri del modello di regressione logistica $\theta = \{(w_c, b_c), c \in (y)\}$ vengono selezionati per ottimizzare la log likelihood condizionata delle etichette dato un set di dati $D = \{(x_i, y_i), i = 1 : n\}$:

$$\theta_* = \arg \max_{\theta} \mathcal{L}(\theta | D) = \arg \max_{\theta} \sum_{i=1}^n \log P(Y = y_i | X = x_i)$$

Tuttavia, la funzione $\mathcal{L}(\theta | D)$ non può essere massimizzata analiticamente. L'apprendimento dei parametri del modello richiede metodi di ottimizzazione numerica.

3.4.2 Discesa del Gradiente

DEFINITION Discesa del Gradiente

La discesa del gradiente è un algoritmo di ottimizzazione iterativo per trovare il minimo di una funzione. Eseguire un passo proporzionale all'opposto del gradiente della funzione nel punto corrente.

Se consideriamo una funzione $f(\theta)$, l'**aggiornamento della discesa** del gradiente, per ogni parametro θ_j , può essere espresso come:

$$\theta_j = \theta_j - \alpha \frac{d}{d\theta_j} f(\theta)$$

La dimensione del passo è controllata dal **tasso di apprendimento** α . Scegliere il tasso di apprendimento α corretto è fondamentale per procedere correttamente verso il minimo:

- Un passo troppo piccolo potrebbe portare a una convergenza estremamente lenta.
- Se il passo è troppo ampio, l'ottimizzatore potrebbe oltrepassare il minimo o addirittura divergere.

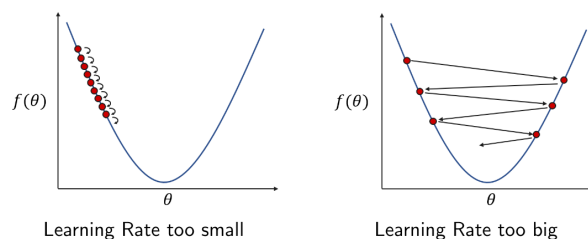


Figura 3.3: Scelta tasso di apprendimento.

Esistono diversi tipi di discesa del gradiente:

- **Batch gradient descent:** ottimizzazione su l'intero batch di dati.
- **Mini batch stochastic gradient descent:** ad ogni step considero solo un piccolo insieme di dati, presi dal training set.
- **Stochastic gradient descent:** caso in cui per ogni step considero solo 1 esempio.

CONVESSITÀ

Se la funzione è convessa, la discesa del gradiente converge al **minimo globale**. Nel caso di funzioni non convesse, può convergere a **minimi locali**.

NOTE FINALI

- In fase di classificazione, la LR è più veloce di NB e LDA. L'apprendimento della LR richiede un'ottimizzazione numerica, che risulta più lenta rispetto a NB.
- Richiede gli stessi parametri degli altri modelli (o meno se $C \ll D$).
- Ha più accuratezza in casi lineari con tante classi.

Capitolo 4

Support Vector Machine

4.1 Classificazione Binaria

Ci troviamo di fronte a un problema di classificazione binaria, dove dobbiamo distinguere i puntini blu da quelli rossi.

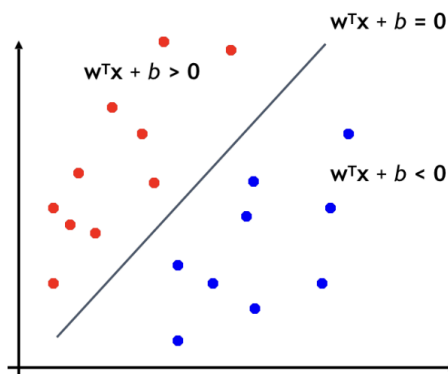


Figura 4.1: Esempio di problema di classificazione binaria.

Bisogna trovare la retta $w^T x + b$ che separi le due classi, ma esistono infinite rette. Va, quindi, trovata quella **ottima**.

Per fare questo, possiamo definire dei **vettori di supporto** e calcolare la loro distanza come:

$$r_i = \frac{w^T x_i + b}{\|w\|}$$

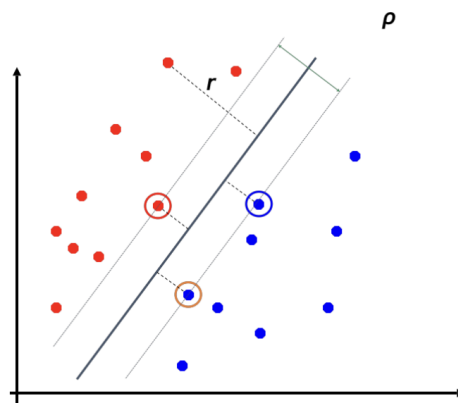


Figura 4.2: Utilizzo vettori di supporto.

L'obiettivo è massimizzare questa quantità per tutti gli x_i . Puntiamo al boundary che offre la maggiore sicurezza.

EXAMPLE Calcolo dei margini

Dato un dataset $D = (x_i, y_i)_{i \in [1, \dots, N]}$ e $y_i \in \{-1, 1\}$ con ρ come margine, vogliamo che:

$$y_i(w^T x_i + b) \geq \rho/2 \quad \forall x_i \in D$$

Per il vettore di supporto x_s l'equazione diventa $w^T x_s + b = \rho/2$, ottenendo quindi questa espressione per il margine:

$$r_s = \frac{w^T x_s + b}{\|w\|} = \frac{\rho}{2\|w\|} = \frac{1}{\|\hat{w}\|}$$

$$\rho = 2r_s = \frac{2}{\|w\|}$$

4.1.1 Problema quadratico SVM

La formula di **massimizzazione** del margine:

$$\arg \max_{w, b} \frac{2}{\|w\|} \quad s.t.$$

$$y_i(w^T x_i + b) \geq 1 \quad \forall (x_i, y_i) \in D$$

Equivalente a quella di **minimizzazione** dei pesi:

$$\arg \min_{w, b} \|w\|^2 \quad s.t.$$

$$y_i(w^T x_i + b) \geq 1 \quad \forall (x_i, y_i) \in D$$

4.1.2 Ottimizzazione Vincolata

In alcuni casi, potrebbe essere necessario ottimizzare problemi condizionati:

$$\min f(x)$$

$$\text{subject to } h_i(x) \geq 0 \quad \text{for } i \in 1, \dots, N$$

DEFINITION Condizioni di Lagrange

La funzione Lagrangiana è:

$$L(x, \alpha) = f(x) - \sum_i^N \alpha_i h_i(x) \quad \text{con } \alpha_i \geq 0$$

Dove:

- Se per x_i , $h(x_i) > 0$ allora $\alpha = 0$ il vincolo si dice **inattivo** e la soluzione è $x^* \mid \nabla f(x) = 0$
- Se per x_i , $h(x_i) = 0$ allora $\alpha > 0$ il vincolo si dice **attivo** e la soluzione è $x^* \mid \nabla f(x) - \alpha^T \nabla h(x) = 0$

Mettendo tutto insieme, troviamo le condizioni di **Karush-Kuhn-Tucker (KKT)**, che costituiscono condizioni necessarie per l'esistenza di una soluzione unica x_* :

$$\nabla f(x^*) - \alpha^T \nabla h(x^*) = 0$$

$$h(x^*) \geq 0$$

$$\alpha_i \geq 0$$

$$\alpha^T h(x) = 0$$

4.1.3 Classificatore SVM

La funzione di classificazione SVM lineare nella forma **primale**, inglobando b nei pesi w con una dimensione in più, è:

$$f(x) = w^T x$$

Invece, nella forma **duale** è:

$$f(x) = \sum_{i=1}^N a_i y_i x_i^T x$$

MARGINI NON SEPARABILI

Se i pattern dei dati non sono separabili mediante un iperpiano lineare, un insieme di **variabili di slack** $\epsilon = \epsilon_1, \epsilon_2, \dots, \epsilon_N$ sono introdotte con $\epsilon_i \geq 0$ facendo diventare i vincoli dell'SVM:

$$y_i(x_i w + b) \geq 1 - \epsilon_i$$

Inoltre, il problema di ottimizzazione diventa:

$$\min \frac{1}{2} \|w\|^2 + C \sum_i \epsilon_i$$

$$\text{subject to } y_i(w^T x_i + b) \geq 1 - \epsilon_i \quad \text{con } \epsilon_i \geq 0$$

4.1.4 SVM non Lineare

Cosa succede se le classi non sono separabili linearmente?

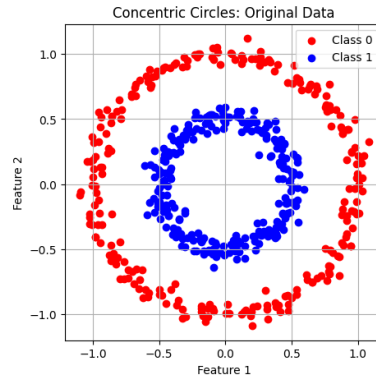


Figura 4.3: Grafico di un SVM non lineare.

La soluzione migliore, è trasformare lo spazio di classificazione in uno in cui il confine è lineare.

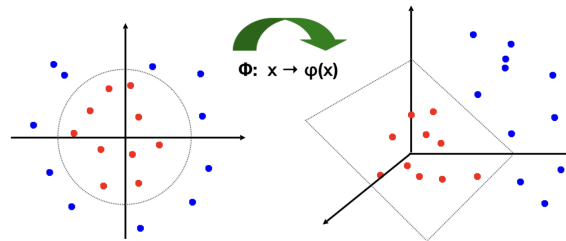


Figura 4.4: Grafico sul cambio di spazio di classificazione.

Potremmo utilizzare una mappatura basata su una funzione non lineare: $\phi : x \rightarrow \phi(x)$

DEFINITION Funzione di Kernel

Una funzione kernel mappa implicitamente i dati in uno spazio ad alta dimensionalità senza la necessità di calcolare esplicitamente ogni $\phi(x)$.

EXAMPLE Definizione funzione kernel

Dato $f(x) = \sum_{i=1}^N a_i y_i (x_i^T x)$:

1. **Mappatura non lineare:** $\phi : x \rightarrow \phi(x)$.
2. **Definire la kernel function:** $\phi(x_i)^T \phi(x_j) = K(x_i, x_j)$.
3. **Sostituzione:** $f(x) = \sum_{i=1}^N a_i y_i K(x_i, x_j)$.

Ci sono, inoltre, vari tipi di kernel:

- **Lineare:** $K(x, x_i) = x^T x_i$, utilizzato dal classificatore SVM classico.
- **Polinomiale:** $K(x, x_i) = (1 + x^T x_i)^p$, espande in maniera polinomiale le dimensioni come $\binom{d+p}{d}$.
- **RBF (Gaussiano):** $K(x, x_i) = \exp(-\frac{\|x-x_i\|^2}{2\sigma^2})$, espande in una dimensione infinita, come $e^x = \sum_{n=1}^{\infty} \frac{x^n}{n!}$.

Capitolo 5

Ensembles

DEFINITION Ensemble

Un ensemble è semplicemente una raccolta di modelli addestrati per svolgere lo stesso compito, può essere costituito da diverse versioni dello stesso modello o da diversi tipi di modelli.

L'output finale di un ensemble di classificatori si ottiene in genere tramite una media o un voto delle previsioni dei diversi modelli che lo compongono.

Un ensemble di modelli diversi che raggiungono tutti prestazioni di generalizzazione simili spesso supera le prestazioni di ciascun singolo modello.

EXAMPLE Proof of Majority Voting

Dato un insieme di funzioni di classificazione binaria $f_k(x)$ per $k = 1, \dots, K$ con una soglia di errore simile $E_{p(x,y)}[y \neq f_k(x)] < 0.5$, allora la maggior parte dei K classificatori dell'ensemble fornirà la risposta corretta su molti esempi per i quali un singolo classificatore commette un errore. Infatti, si commette un errore solo quando $\frac{k}{2} + 1$ classificatori fanno una predizione sbagliata. Gli errori seguono la distribuzione binomiale cumulativa:

$$P(x \leq k) = \sum_{i=0}^k \binom{K}{i} \alpha^i (1 - \alpha)^{K-i}$$

5.1 Bagging

Il Bagging prende un singolo set di addestramento T_r ed effettua un sottocampionamento casuale da esso per K volte per formare K set di addestramento $T_{r_1}, \dots, T_{r_K}$. Ciascuno di questi set di addestramento viene utilizzato per addestrare un'istanza diversa dello stesso classificatore, ottenendo K funzioni di classificazione $f_1(x), \dots, f_K(x)$.

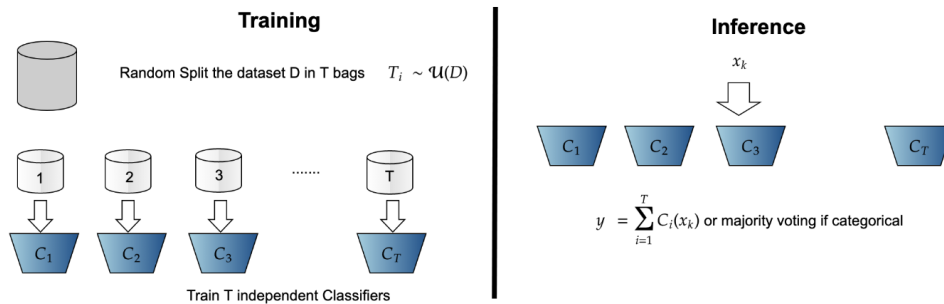


Figura 5.1: Approccio del Bagging

EXAMPLE Dimostrazione

Supponiamo che ogni classificatore abbia come funzione $y = f_i(x) + e_i$ dove e è l'errore. L'errore medio di M classificatori sul dataset D è:

$$\mathcal{E}_{AV} = \frac{1}{M} \sum_{i=1}^M E_D[e_i]$$

Invece, la predizione col bagging:

$$y_{\text{bagged}} = \frac{1}{M} \sum_{i=1}^M f_i(x) + e_i(x)$$

Quindi, l'errore medio del classificatore bagged è:

$$\mathcal{E} = E_D \left[\frac{1}{M} \sum_{i=1}^M e_i(x) \right]$$

5.2 Boosting

Il boosting parte da un dataset D e addestra sequenzialmente dei classificatori f_i , concentrandosi sugli errori commessi dai classificatori precedenti. Addestrato per correggere con successo gli errori dei modelli precedenti.

EXAMPLE Approccio

Assegna a ogni $x_k \in D$ peso uguale $w_k = \frac{1}{N}$, poi successivamente iterare con i da 1 a M , quindi il numero di stage del boosting:

1. Genera un nuovo dataset D_i da D usando i pesi $\{w_k^i\}_{k=1}^N$ come probabilità di scelta per ogni record x .
2. Allena l' i -esimo classificatore f_i su D_i e misura l'accuratezza α_i .
3. Se x_k è stato classificato erroneamente, incrementane il peso per la fase successiva $(w_i + 1)$ e normalizza nuovamente i pesi.

La decisione finale equivale a:

$$y(x_{\text{new}}) = \text{sign} \left(\sum_i^M \alpha_i f_i(x_{\text{new}}) \right)$$

Ogni classificatore nella catena di boosting deve avere un'accuratezza superiore a 0.5.

Agli elementi classificati erroneamente viene attribuito un peso maggiore e aumenta la probabilità che vengano campionati man mano che la catena si allunga. Catene più lunghe tendono ad aumentare l'accuratezza, ma anche il rischio di overfitting.

5.3 AdaBoost

L'obiettivo è eliminare il campionamento dagli algoritmi di boosting convenzionali, sfruttando il peso del campione nella valutazione dell'errore della singola fase.

L'errore di un classificatore alla fase k è:

$$\epsilon_k = \sum_{i=1}^N w_k^i \mathbb{I}(f_k(x^i) \neq y^i)$$

Dopo ogni fase degli algoritmi di boosting, i pesi dei campioni vengono modificati, tramite la **trust**

$\alpha_k = \log \frac{1-\epsilon_k}{\epsilon_k}$:

- Il peso degli elementi classificati erroneamente viene aumentato: $w_{k+1}^i = w_k^i \exp^{+\alpha_k}$.
- Il peso degli elementi classificati correttamente viene diminuito: $w_{k+1}^i = w_k^i \exp^{-\alpha_k}$.

Ciò significa che, in ogni fase, gli elementi classificati in modo errato sono più importanti per il classificatore corrente e ciascun classificatore, in ogni fase, deve comunque raggiungere un'accuratezza superiore a 0.5, in termini di accuratezza pesata, altrimenti la catena si riavvia.

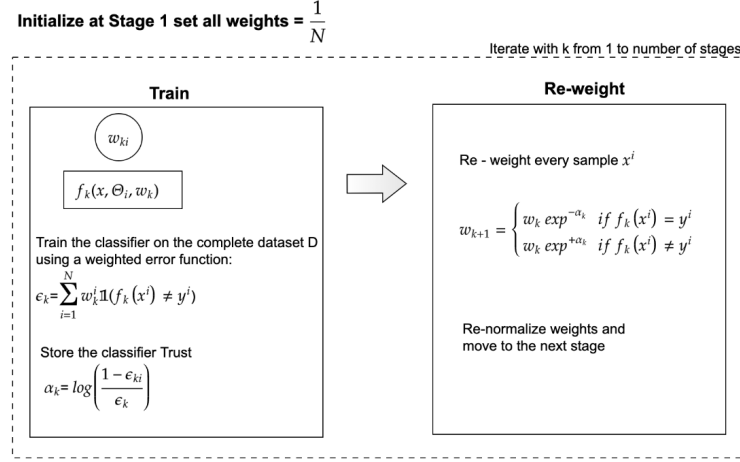


Figura 5.2: Approccio del Bagging

La regola decisionale finale è una somma ponderata delle risposte individuali:

$$y^i = \text{sign}\left(\sum_{k=1}^M \alpha_k f_k(x^i)\right)$$

Capitolo 6

Random Forest

6.1 Alberi Binari

Gli alberi binari sono una struttura dati in cui ogni nodo padre ha messo due nodi figli, destro e sinistro. I nodi si dividono in:

- **Split nodes:** nodi di decisione.
- **Leaf nodes:** nodi finali con la soluzione.

Inoltre, ci sono alcuni parametri importanti da considerare:

- **Vettore di feature:** $v \in \mathbb{R}^N$.
- **Funzione di split:** $f_n(v) : \mathbb{R} \rightarrow \mathbb{R}^N$.
- **Thresholds:** $t_n \in \mathbb{R}$.
- **Classificazione:** $P_n(c)$.

Ogni decisione presa nell'albero rappresenta una retta.

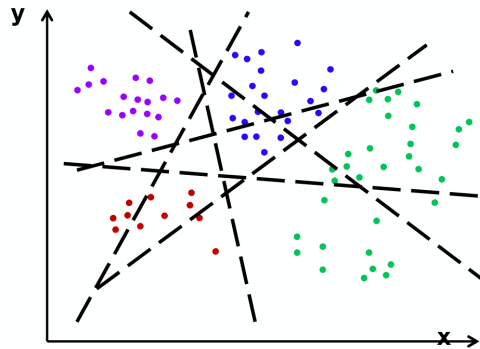


Figura 6.1: Dimostrazione grafica delle scelte in un albero.

6.2 Ottimizzazione

Per prima cosa bisogna definire le due label, destra e sinistra:

1. **Left split:** $I_l = \{i \in I_n \mid f(v_i) < t\}$.
2. **Right split:** $I_r = I_n / I_l$.

Dove $t \in (\min_i f(v_i), \max_i f(v_i))$. L'obiettivo è massimizzare l'**information gain**, scegliendo correttamente la funzione f e il threshold t . Per fare questo, è necessario minimizzare l'entropia, che si calcola come:

$$E = - \sum_i^{nLabels} h_i \log_2(h_i)$$

Ottenendo:

$$\Delta E = -\frac{|I_l|}{|I_n|}E(I_l) - \frac{|I_r|}{|I_n|}E(I_r)$$

QUANTE FEATURE E THRESHOLD USARE?

Ci sono diversi casi:

- **Solo uno:** sarebbe troppo randomico.
- **Pochi:** addestramento rapido ma rischio di underfitting e forse eccessiva profondità.
- **Tanti:** addestramento più lento e possibile overfitting.

6.3 Foresta

Una foresta è un insieme di alberi decisionali.

Utilizziamo la media delle classificazioni con alberi decisionali. La classificazione risulta una normalizzazione rispetto al numero di alberi:

$$P(c|v) = \frac{1}{T} \sum_{t=1}^T P_t(c|v)$$

Ci sono tre tipi di foreste:

1. **Classificazione**, massimizzare l'information gain:

$$I = H(S) - \sum_{i=1}^2 \frac{|S_i|}{|S|} H(S_i)$$

Serve a classificare le classi.

2. **Regressione**, massimizzare l'information gain:

$$Err(S) - \sum_{i=1}^2 \frac{|S_i|}{|S|} Err(S_i)$$

Dove:

$$Err(S) = \sum_{j \in S} (y_j - y(x_j))^2$$

Serve per predire valori reali.

3. **Clustering**, ci sono due possibilità, minimizzare l'**imbalance**:

$$B = |\log |S_1| - \log |S_2||$$

Oppure massimizzare la likelihood della Gaussiana:

$$T = |\Lambda_S| - \sum_{i=1}^2 \frac{|S_i|}{|S|} |\Lambda_{S_i}|$$

Serve per predire una somiglianza fra classi.

Capitolo 7

Clustering

DEFINITION Clustering

Organizzare i dati in classi in modo tale da avere:

- Elevata somiglianza intra-classe
- Bassa somiglianza inter-classe

Individua le etichette di classe e il numero di classi direttamente dai dati.
Più informalmente, individua raggruppamenti naturali tra gli oggetti.

Ci sono due tipi di clustering:

1. **Gerarchico:** gli oggetti vengono raggruppati in un albero gerarchico.
2. **Partizioni:** gli oggetti vengono divisi in base a diversi criteri, in diversi gruppi.

7.1 Distanza e Similiarità

Nei metodi di clustering c'è differenza fra similiarità e distanza.

DEFINITION Distanza

La metrica di distanza deve rispettare queste proprietà:

- **Simmetria:** $D(A, B) = D(B, A)$
- **Autosimiliarità:** $D(A, A) = 0$
- **Positività:** $D(A, B) = 0$ se $A = B$
- **Inegualità triangolare:** $D(A, B) \leq D(A, C) + D(B, C)$

7.2 Clustering Gerarchico

Non si possono testare tutti gli esiti, quindi si adottano due strategie:

1. **Bottom-up (agglomerativo):** partendo da ogni elemento nel proprio cluster, individua la coppia migliore da unire in un nuovo cluster. Ripeti l'operazione finché tutti i cluster non saranno fusi insieme.
2. **Top-Down (divisivo):** partendo da tutti i dati in un unico cluster, si considerino tutti i modi possibili per suddividere il cluster in due parti. Si scelga la suddivisione migliore e si operi ricorsivamente su entrambe le parti.

7.2.1 Strategia di Merging

Per unire i cluster tra loro, ho bisogno di trovare un modo per minimizzare la distanza.

Ci sono vari approcci per calcolare la distanza tra due cluster:

- **Single linkage:** distanza calcolata fra due oggetti scelti tra i due cluster.
- **Complete linkage:** distanza massima calcolata fra due oggetti scelti tra i due cluster.
- **Group Average Linkage:** distanza media calcolata tra ogni oggetto dei due cluster.
- **Wards Linkage:** si minimizza la varianza tra gli oggetti del cluster, non si calcola la distanza.

7.3 Clustering non Gerarchico

Ogni istanza viene assegnata esattamente a uno di K cluster tra loro disgiunti.

Viene prodotto un unico insieme di cluster, l'utente deve solitamente specificare il numero desiderato di cluster K .

DEFINITION Funzione Obiettivo K-Means

Dato un insieme di punti $(x_1, x_2, \dots, x_n)$ e k insiemi $S = \{S_1, S_2, S_3, \dots, S_K\}$, la sua funzione obiettivo è:

$$\arg \min_S \sum_{i=1}^k \sum_{x \in S_i} \|x - \mu\|^2 = \arg \min_S \sum_{i=1}^k |S_i| \text{Var} S_i = \arg \min_S \sum_{i=1}^k \sum_{x, y \in S_i} \|x - y\|^2$$

Dove μ_i è la media dei punti in S_i .

Con questa formula possiamo classificare tutti gli oggetti nel relativo cluster.

ALCUNE OSSERVAZIONI SU K-MEANS

- **Vantaggi:** Relativamente efficiente, $O(tkn)$, quando $k, t \ll n$. Spesso termina su un minimo locale.
- **Svantaggi:** Applicabile solo quando si conosce la media, non usabile su cluster con forme non convesse e bisogno di specificare il numero di cluster in anticipo. Inoltre, è sensibile ai rumori e a outliers.

7.4 Grafi

DEFINITION Grafo

Un grafo è formato da un insieme di **vertici** V e un insieme di **archi** ϵ .

Il grafo è **orientato** se gli archi sono orientati, altrimenti è **non orientato**. Inoltre, un grafo si dice **bipartito** se può essere partizionato in due insiemi di vertici disgiunti, senza collegamenti tra i due.

L'obiettivo è trovare un taglio che divida il grafo in due cluster, il nostro obiettivo è trovare il taglio migliore. Per farlo è utile conoscere:

- **Prodotto scalare e funzione reale vertici** $f : V \rightarrow \mathbb{R}$. Dato un vettore $f(f_1, f_2, \dots, f_n) \in \mathbb{R}^n$ con $n = |V|$:

$$\langle f, g \rangle_{\mathcal{H}(V)} := \sum_{x_i \in V} f(x_i)g(x_i)$$

- **Prodotto scalare e funzione reale archi**, $F : E \rightarrow \mathbb{R}$. Dato un vettore $F(f_1, f_2, \dots, f_n) \in \mathbb{R}^n$ con $n = |E|$:

$$\langle F, G \rangle_{\mathcal{G}(E)} := \sum_{x_i, x_j \in E} F(x_i, x_j)G(x_i, x_j)$$

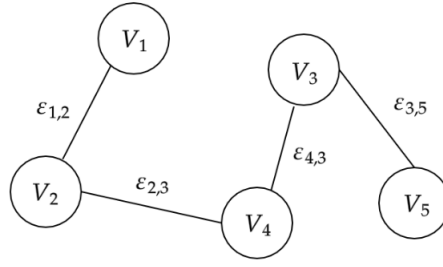


Figura 7.1: Esempio di grafo orientato.

7.4.1 Operatori

DEFINITION Differenza Pesata

La differenza pesata (o derivata pesata sul grafo) di f lungo un arco orientato $(x_i, x_j) \in E$ è:

$$\partial_{x_j} f(x_i) := \sqrt{w(x_i, x_j)}(f(x_j) - f(x_i))$$

DEFINITION Gradiente Pesato

Sulla base della nozione di differenze pesate, si definisce l'operatore gradiente pesato sui grafi:

$$\nabla_w : \mathcal{H}(V) \rightarrow \mathcal{H}(E) \quad \nabla_w : \mathcal{H}(V) \rightarrow \mathcal{H}(E)$$

Come:

$$(\nabla_w f)(x_i, x_j) = \partial_{x_j} f(x_i)$$

DEFINITION Adjoint

Due operatori, si definiscono adjoint, quando si possono invertire l'ordine dell'operazione, per invertire l'operatore desiderato. L'operatore adjoint $\nabla_w^* : \mathcal{H}(E) \rightarrow \mathcal{H}(V)$ del gradiente pesato è un operatore lineare definito da:

$$\langle \nabla_w f, G \rangle_{\mathcal{H}(E)} = \langle f, \nabla_w^* G \rangle_{\mathcal{H}(V)} \quad \forall f \in \mathcal{H}(V), G \in \mathcal{H}(E)$$

Affinchè:

$$(\nabla_w^* F)(x_i) = \sum_{x_j \sim x_i} \sqrt{w(x_i, x_j)}(F(x_j, x_i) - F(x_i, x_j))$$

L'operatore adjoint negativo $-\nabla_w^*$ è chiamato **divergenza**, che misura il flusso netto in uscita di una funzione di arco in ciascun vertice del grafo.

DEFINITION Laplaciano

È un operatore utile, per il calcolo del taglio ottimo, si ottiene facendo la divergenza del gradiente di una funzione:

$$(\text{div}_w(\nabla_w f))(x_i)$$

Dato un grafo, possiamo ottenere il laplaciano in questo modo:

$$L = D - W$$

Dove:

- $\mathbf{W}$ è una matrice $i \times j$ con w_{ij} è il peso che collega i e j .
- $\mathbf{D}$ è una matrice diagonale, in cui nella diagonale ha la somma della riga.

7.5 Partizione Dati tramite Grafi

Dato un dataset $D = \{x_i\}_{i=1}^N$ con $x_i \in \mathcal{R}^D$, possiamo costruire un grafo nel quale:

- Ogni vertice V è un datapoint $V_i = x_i$.
- Il peso di ogni arco w_{ij} è la similiarità fra i due datapoint $(V_i, V_j) = (x_i, x_j)$.

7.5.1 Bi-Partizione del Grafo

Per ottenere due insiemi di vertici disgiunti bisogna tagliare gli archi che li connettono, avendo un costo:

$$\text{CUT}(A, B) = \sum_{i \in A, j \in B} w_{ij}$$

Ossia, la sommatoria della somiglianza tra gli oggetti dei due cluster.

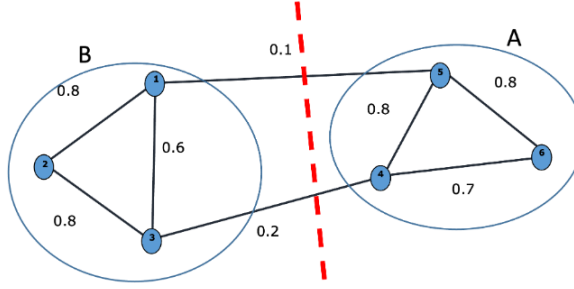


Figura 7.2: Esempio di bi-partizionamento di un grafo.

Esistono due approcci di clustering dei grafi:

1. **Minimum cut:** $\text{mincut}(A, B)$.
2. **Normalized cut:** $\text{minNcut}(A, B) = \frac{\text{cut}(A, B)}{\text{vol}(A)} + \frac{\text{cut}(A, B)}{\text{vol}(B)}$.

7.5.2 Minimum Cut

Risolvere il problema del taglio minimo equivale a trovare un vettore discreto p dove:

$$p = \begin{cases} +1 & \text{se } V_i \in A \\ -1 & \text{se } V_i \in B \end{cases}$$

è un problema NP completo. Possiamo per risolverlo usando $f : V \rightarrow \mathcal{R}$ come rilassamento di p . Riscrivendo l'equazione otteniamo:

$$\text{mincut}(A, B) \arg \min_f \sum_E w_{i,j} (f_i - f_j)^2$$

O in forma matriciale:

$$f^T L f$$

Che equivale anche all'**Energia di Dirichlet**, usata per calcolare la regolarità di una funzione:

$$\sum_E \|\nabla_w f\|^2$$

OTTIMIZZAZIONE

Risolvere i problemi di taglio minimo equivale a trovare un campo di vertici continuo f che minimizzi $f^T L f$ che equivale a minimizzare l'energia di Dirichlet.

THEOREM Teorema di Rayleigh

La soluzione Mincut f^* è:

- Il secondo autovettore più piccolo di L .
- Il valore di taglio è il secondo autovalore più piccolo di L .

Tutte le possibili soluzioni non sovrapposte di $\arg \min_f \sum_E \|\nabla_w f\|^2$ equivalgono agli autovettori del laplaciano L ordinati per autovalori crescenti.

Infine, per partizionare un grafo in k cluster, posso usare ricorsivamente la tecnica del bi-partizionamento.

Capitolo 8

Dimensionality Reduction

L'obiettivo, è quello di trattare un dataset D-dimensionale, come uno K-dimensionale, con $K \ll D$.

8.1 Multi Dimensional Scaling (DMS)

Assegnare gli elementi in uno spazio K-dimensionale, cercando di minimizzare lo stress.

DEFINITION Stress

Quando il nuovo punto mantiene le informazioni del vecchio:

$$\text{stress} = \sqrt{\frac{\sum_{i,j} (\hat{d}_{ij} - d_{ij})^2}{\sum_{i,j} d_{ij}^2}}, \quad d_{ij} = |o_j - o_i|$$

Dove d rappresenta la distanza tra i e j .

Si vuole minimizzare lo stress per ogni punto, ma ha una complessità computazionale di:

- $O(N^2)$ a running time.
- $O(N)$ per aggiungere un oggetto.

Ci sono due riduzioni principali:

- **Isometrico**, mantiene la distanza:

$$D'(F(i), F(j)) = D(i, j)$$

- **Contrattivo**, diminuisce la distanza:

$$D'(F(i), F(j)) \leq D(i, j)$$

8.1.1 PCA

Una tecnica di riduzione lineare, divisa in diverse fasi:

1. Creazione di una matrice $X = N \times d$.
2. Sottrarre a ogni x_n il valore x_{medio} della matrice X .
3. Calcolare la covarianza Σ della matrice X . Dato $\Sigma = AA^T$ dove $A = [r_1, \dots, r_m]$.
4. Calcolare gli autovettori e gli autovalori della covarianza Σ . Dato:
 - (a) **Autovalore**: $Av = \lambda v$.
 - (b) **Autovettore**: $AA^T(Av_i) = \mu_i(Av_i)$ oppure $A^T Av_i = \mu_i v_i$.
5. Scegliere gli M autovettori più grandi, nello specifico ogni coppia di autovettori deve avere covarianza zero.
6. Proiettare i dati in uno spazio K-dimensionale: $p = U^T(x - \mu)$.

NOTA BENE

Se ci sono grandi quantità di dati, come per esempio un numero di immagini maggiore di 10^4 , si può notare che la covarianza ha una dimensione di d^2 , quindi si avrebbe $d \geq 10^4$ con $\Sigma \geq 10^8$.

8.1.2 Riduzione non Lineare**DEFINITION** Manifold

Un manifold è uno spazio topologico localmente euclideo. In generale, qualsiasi oggetto che sia quasi piatto su piccola scala.

DEFINITION Embedding

Un embedding è una rappresentazione di un oggetto topologico, di una varietà, di un grafo, di un campo, in un determinato spazio, tale da preservarne la connettività o le proprietà algebriche. Per esempio, gli insiemi dei numeri reali, razionali e interi.

DEFINITION Geodesic

La curva più breve su un manifold che congiunge due punti del manifold stesso. Ha una sua lunghezza ed è utile perché la distanza euclidea tende a non essere molto buona.

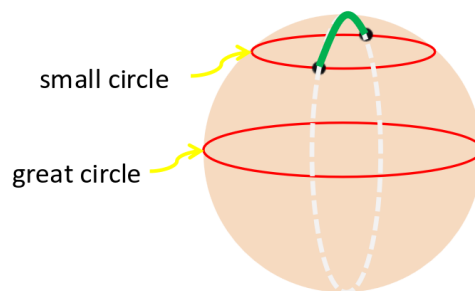


Figura 8.1: Rappresentazione di un geodesic in una sfera.

8.1.3 LLE e Laplacian Eignmap

Algoritmo basato su grafi e ha tre fasi:

1. Trovare K vicini per ottenere un manifold, rappresentato con una laplaciana e tramite un grafico dei punti vicini.
2. Stimare le proprietà del manifold guardando i punti vicini.
3. Cercare un embedding globale che mantiene le proprietà trovate.

Per costruire la lagrangiana, abbiamo bisogno di una matrice W , quella dei pesi, che definisce quando è importante il collegamento tra due nodi. Ci sono due modi per calcolare la matrice dei pesi:

1. **Metodo semplice:** 0 se non c'è collegamento, 1 se è presente.
2. **Heat Kernel:** considera l'importanza dei collegamenti presenti $A_{ij} = e^{-\frac{\|x_i - x_j\|^2}{t}}$.

Possiamo trovare il manifold ottimale, ottimizzando:

$$\arg \min_Y \text{trace}(Y'LY)$$

La soluzione sono i primi m autovettori.